Lab 7 - Naive Bayes Classification (Gaussian Naive Bayes)

NAME: ADITYA BHENDE
CLASS: C3
ROLL NO: 28
BATCH: C2

```
from google.colab import files
uploaded = files.upload()
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, classification_report, confusion_matrix,
                             ConfusionMatrixDisplay)
```

```
df = pd.read_excel("Raisin_Dataset.xlsx")
```

```
print("First 5 Rows:")
print(df.head())
print("\nLast 5 Rows:")
print(df.tail())
print("\nDataset Shape:", df.shape)
print("\nDataset Summary Statistics:")
print(df.describe())
print("\nDataset Information:")
df.info()
print("\nMissing Values Count:")
print(df.isnull().sum())
```

```
       Extent  Perimeter  Class
895  0.668793   1129.072  Besni
896  0.636476   1214.252  Besni
897  0.741099   1292.828  Besni
898  0.658798   1258.548  Besni
899  0.632020   1272.862  Besni

Dataset Shape: (900, 8)

Dataset Summary Statistics:
                Area  MajorAxisLength  MinorAxisLength  Eccentricity  \
count     900.000000       900.000000       900.000000    900.000000
mean    87804.127778       430.929950       254.488133      0.781542
std     39002.111390       116.035121        49.988902      0.090318
min     25387.000000       225.629541       143.710872      0.348730
25%     59348.000000       345.442898       219.111126      0.741766
50%     78902.000000       407.803951       247.848409      0.798846
75%    105028.250000       494.187014       279.888575      0.842571
max    235047.000000       997.291941       492.275279      0.962124
```

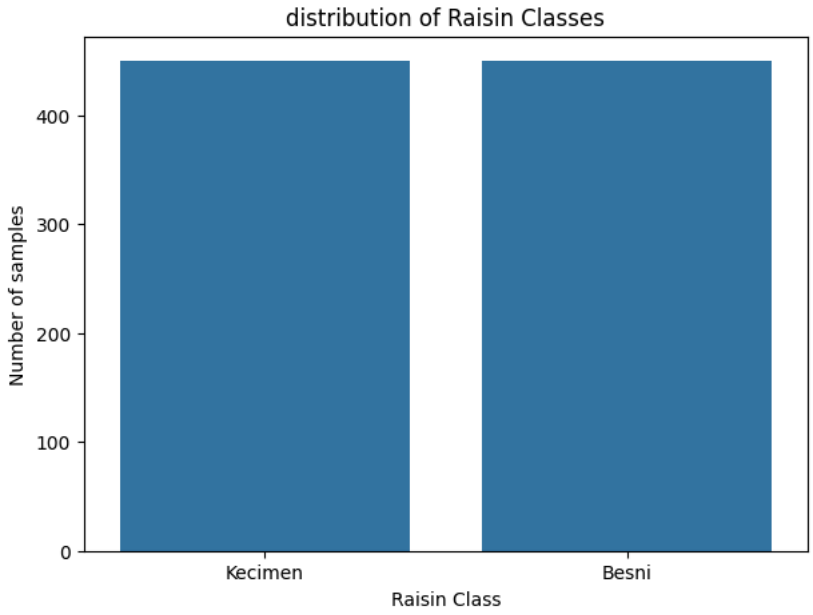
```
 5   Extent           900 non-null    float64
```

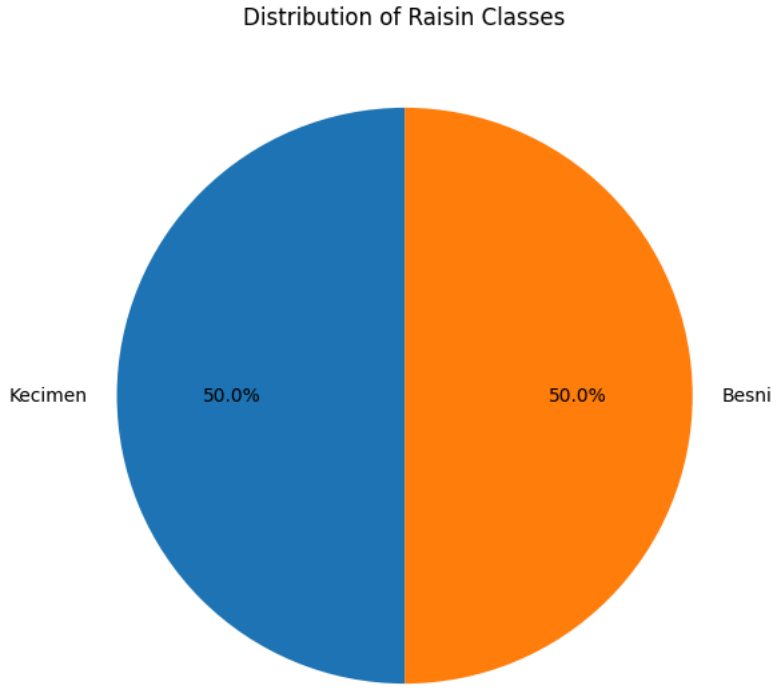
```
 6   Perimeter        900 non-null    float64
 7   Class            900 non-null    object
dtypes: float64(5), int64(2), object(1)
memory usage: 56.4+ KB

Missing Values Count:
Area               0
MajorAxisLength    0
MinorAxisLength    0
Eccentricity       0
ConvexArea         0
Extent             0
Perimeter          0
Class              0
dtype: int64
```

```
plt.figure(figsize=(7, 5))
sns.countplot(data=df, x="Class")
plt.title("distribution of Raisin Classes")
plt.xlabel("Raisin Class")
plt.ylabel("Number of samples")
plt.show()

class_counts = df['Class'].value_counts()
plt.figure(figsize=(7, 7))
plt.pie(class_counts, labels=class_counts.index, autopct='%1.1f%%', startangle=90)
plt.title('Distribution of Raisin Classes')
plt.show()
```






```
numerical_features = df.select_dtypes(include=np.number).columns
print("Numerical features:", numerical_features.tolist())

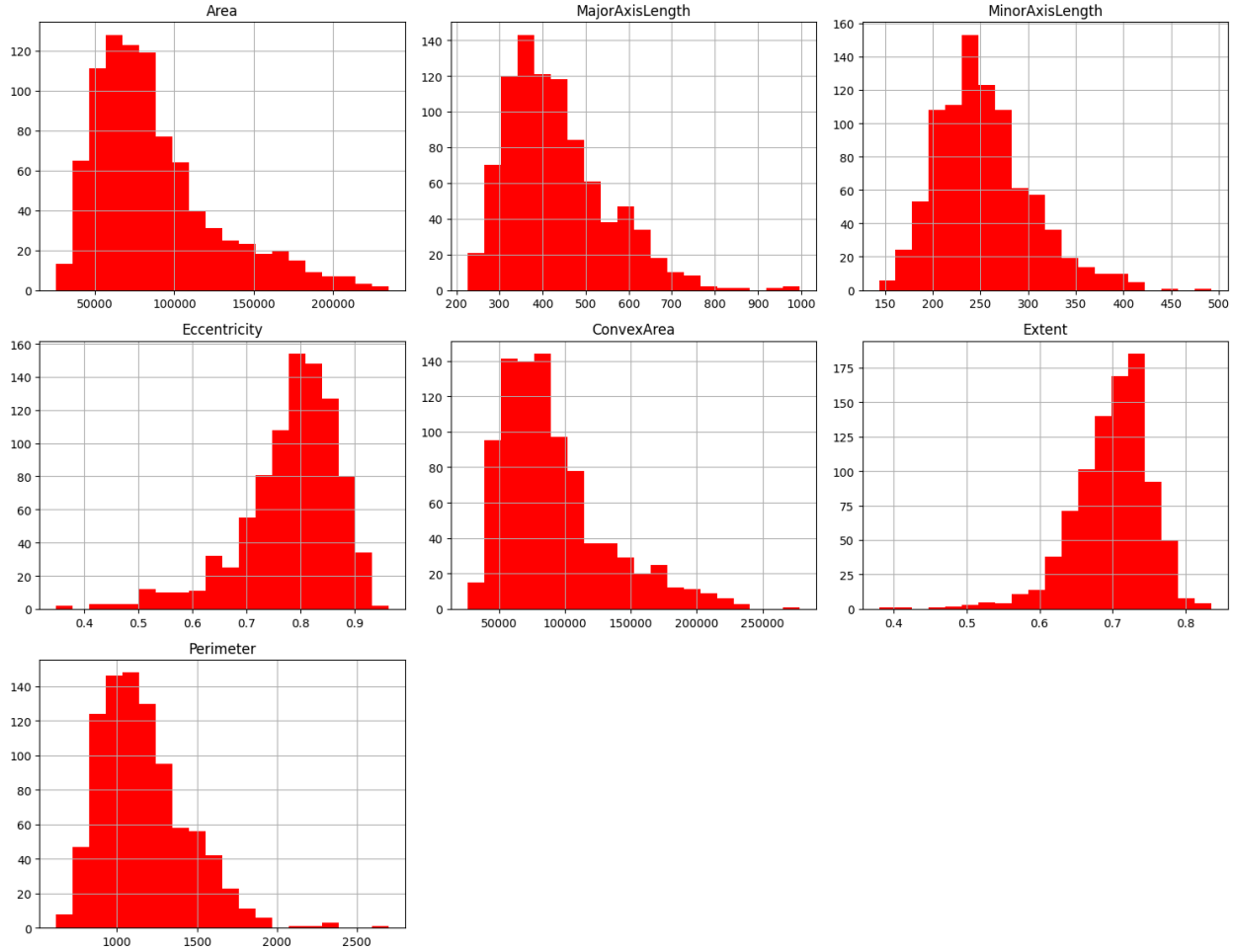

df[numerical_features].hist(figsize=(15, 12), bins=20, color='red')
plt.suptitle("distribution of numerical features", fontsize=16)
plt.tight_layout()
plt.show()

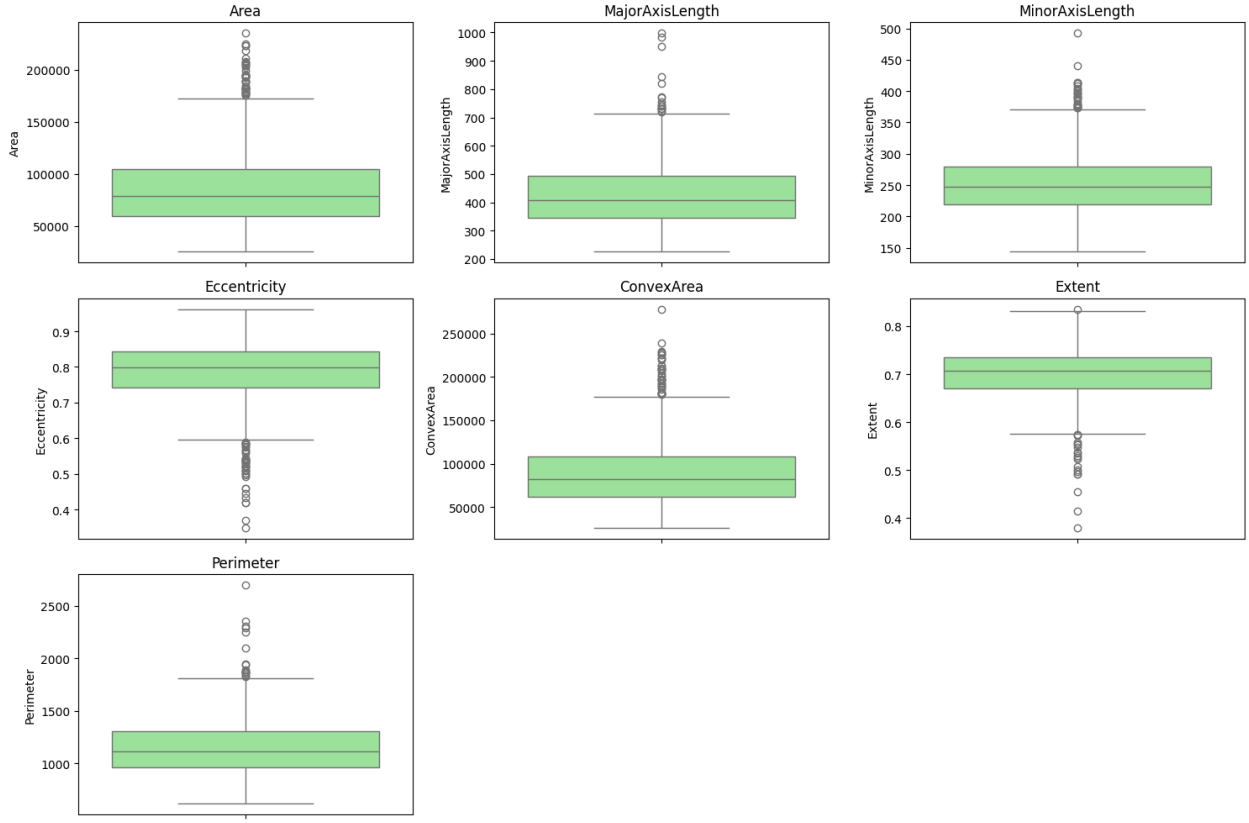

plt.figure(figsize=(15, 10))
for i, column in enumerate(numerical_features, 1):
    plt.subplot(3, 3, i)
    sns.boxplot(y=df[column], color='lightgreen')
    plt.title(column)
plt.tight_layout()
plt.show()

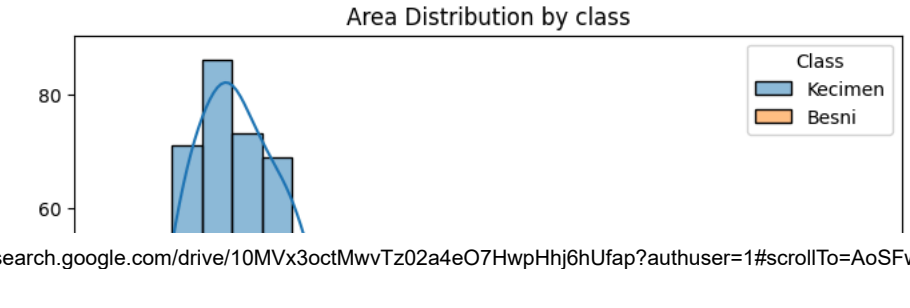

for column in numerical_features:
    plt.figure(figsize=(8, 5))
    sns.histplot(data=df, x=column, kde=True, hue="Class", bins=25)
    plt.title(f"{column} Distribution by class")
    plt.show()
```

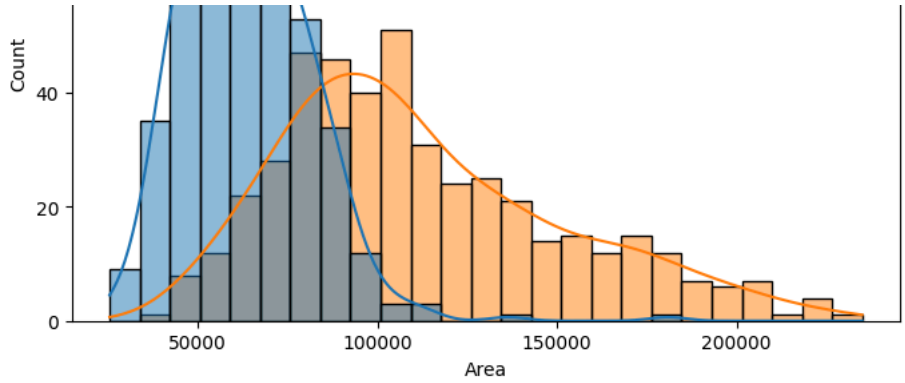
```
Numerical features: ['Area', 'MajorAxisLength', 'MinorAxisLength', 'Eccentricity', 'ConvexArea', 'Extent', 'Perimeter']
```

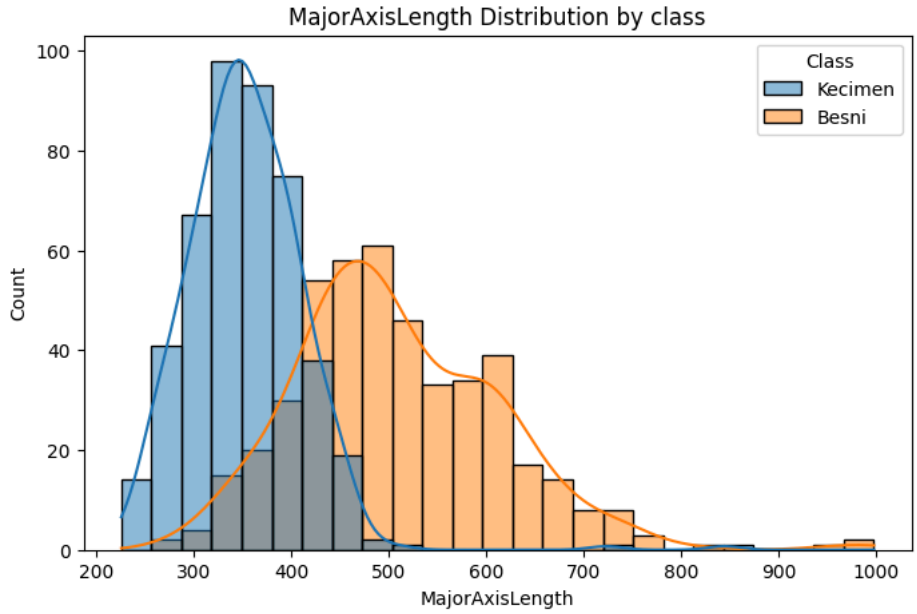
distribution of numerical features

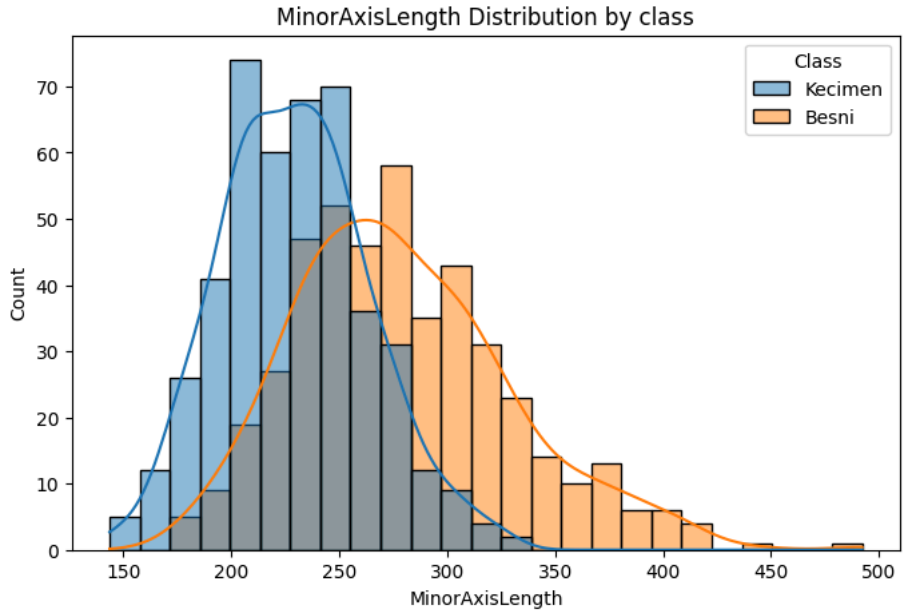


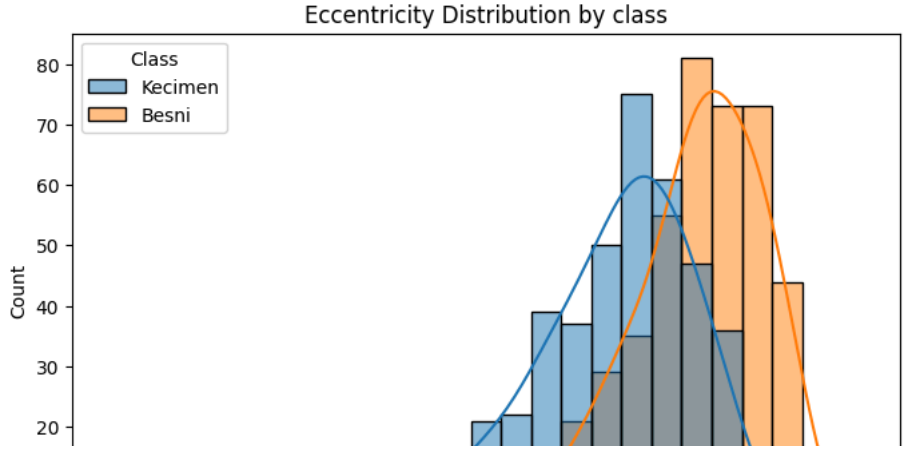


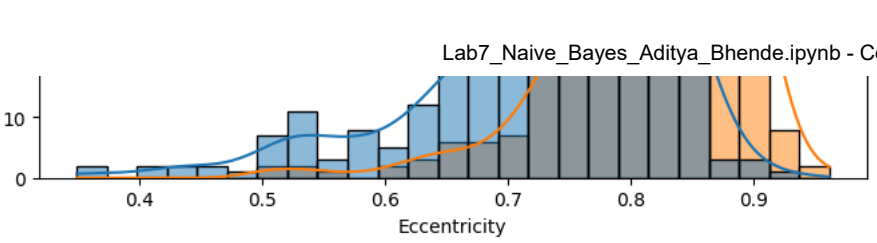


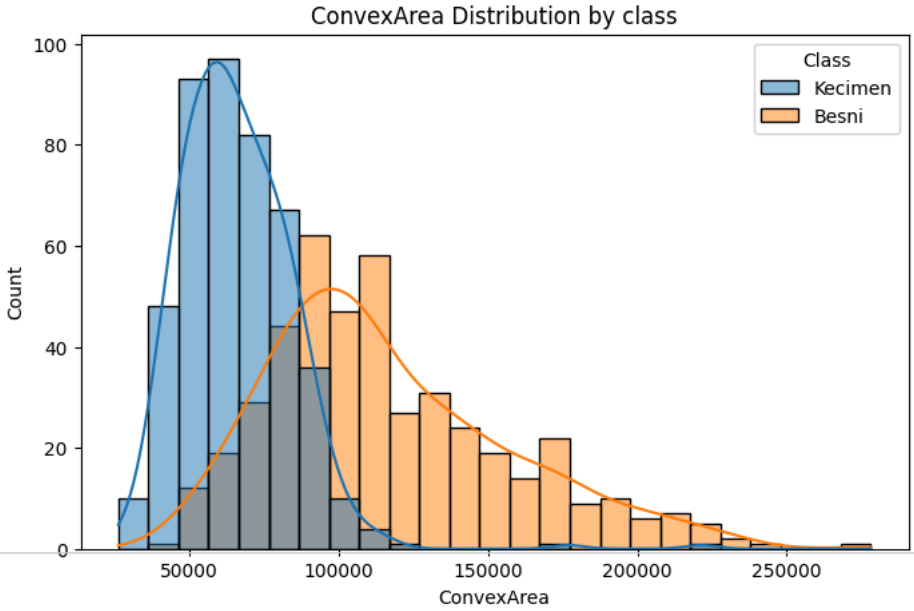

Count
0
20
40
50000
100000
150000
200000
Area

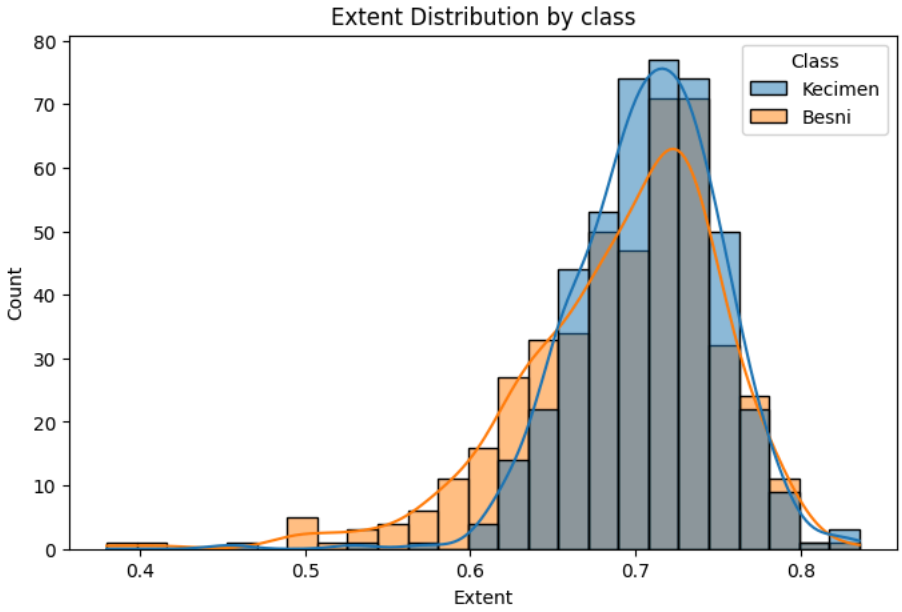

MajorAxisLength Distribution by class
Class
Kecimen
Besni
Count
0
20
40
60
80
100
200
300
400
500
600
700
800
900
1000
MajorAxisLength

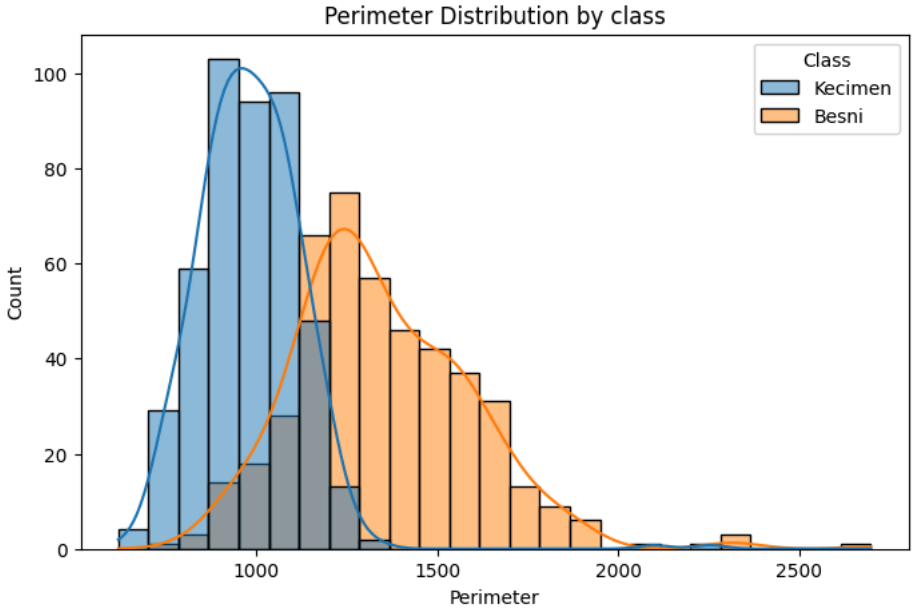

MinorAxisLength Distribution by class
Class
Kecimen
Besni
Count
0
10
20
30
40
50
60
70
150
200
250
300
350
400
450
500
MinorAxisLength


Eccentricity Distribution by class
Class
Kecimen
Besni
Count
20
30
40
50
60
70
80


10
0
0.4
0.5
0.6
0.7
0.8
0.9
Eccentricity


ConvexArea Distribution by class
Class
Kecimen
Besni
Count
100
80
60
40
20
0
50000
100000
150000
200000
250000
ConvexArea


Extent Distribution by class
Class
Kecimen
Besni
Count
80
70
60
50
40
30
20
10
0
0.4
0.5
0.6
0.7
0.8
Extent


Perimeter Distribution by class
Class
Kecimen
Besni
Count
100
80
60
40
20
0
1000
1500
2000
2500
Perimeter

```
correlation_matrix = df[numerical_features].corr()
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="nipy_spectral_r")
plt.title("correlation matrix")
plt.show()

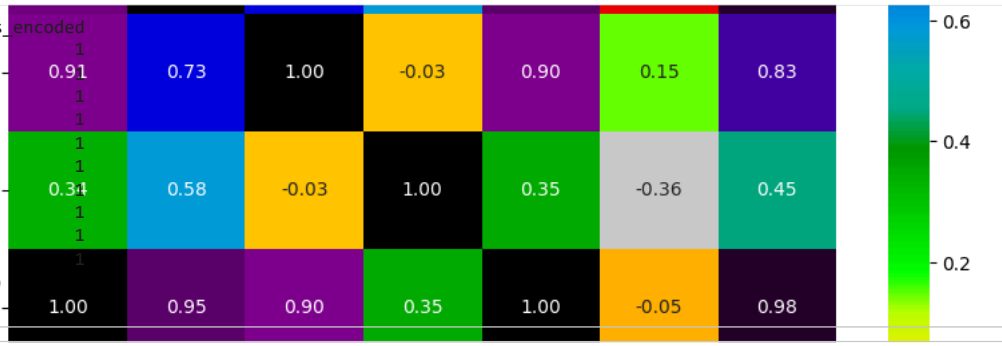

plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x="Area", y="Perimeter", hue="Class",
                palette={'Kecimen': 'Green', 'Besni': 'Red'})
plt.title("Area vs Perimeter")
plt.show()
```

correlation matrix

```
label_encoder = LabelEncoder()
df["Class_encoded"] = label_encoder.fit_transform(df["Class"])
print(df[["Class", "Class_encoded"]].head(10))

X = df[numerical_features]
y = df["Class_encoded"]
print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
     Class  Class_encoded
0  Kecimen              1
1  Kecimen              1
2  Kecimen              1
3  Kecimen              1
4  Kecimen              1
5  Kecimen              1
6  Kecimen              1
7  Kecimen              1
8  Kecimen              1
9  Kecimen              1
X shape: (900, 7)
y shape: (900,)
```

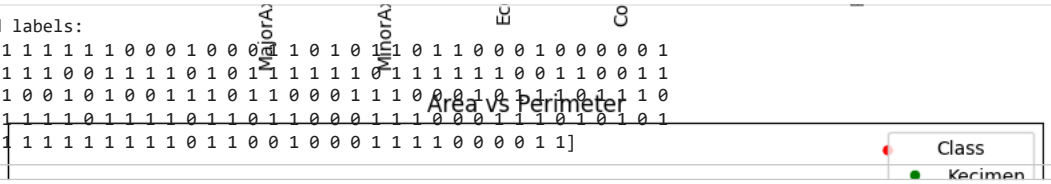



```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 720
Testing samples: 180
```

```
gnb = GaussianNB()
gnb.fit(X_train, y_train)
y_pred = gnb.predict(X_test)
print("predicted labels:")
print(y_pred)
```

```
predicted labels:
[0 1 1 0 1 1 1 1 1 1 0 0 0 1 0 0 0 1 1 0 1 0 1 1 0 1 1 0 0 0 1 0 0 0 0 0 1
 0 1 1 1 1 1 1 0 0 1 1 1 1 0 1 0 1 1 1 1 1 1 0 1 1 1 1 1 1 0 0 1 1 0 0 1 1
 0 0 1 1 1 0 0 1 0 1 0 0 1 1 1 0 1 1 0 0 0 1 1 1 0 0 0 1 0 1 1 1 0 1 1 1 0
 0 1 1 0 1 1 1 1 0 1 1 1 1 0 1 1 0 1 1 0 0 0 1 1 1 0 0 0 1 1 1 0 1 0 1 0 1
 1 1 1 0 1 1 1 1 1 1 1 1 0 1 1 0 0 1 0 0 0 1 1 1 1 0 0 0 0 1 1]
```




```
comparison = pd.DataFrame({"Actual": y_test, "Predicted": y_pred})
print(comparison.head(20))
```

```
     Actual  Predicted
70        1          0
827       0          1
231       1          1
588       0          0
39        1          1
731       0          1
251       1          1
110       1          1
72        1          1
86        1          1
139       1          0
533       0          0
801       0          0
```

